Hochschule Fulda
University of Applied Sciences

2 Results of coding session and systematic unit testing

This study is submitted by:

Sasi Varun Ramachandran, student id: 1569230

Weather station – Station status monitoring unit

2.1 Results of coding session

Low-Level Test Cases:

These test cases were implemented using the **Test-First approach** to validate the station status calculation based on device error rates and ages.

1. **TC-L-01: No Critical Devices (Green Status)**
   - **Input:**
     - Error Rates: [0, 0, 0]
     - Ages: [2, 5, 7]
   - **Expected Output:** "Green"
   - **Result:** Passed
2. **TC-L-02: One Critical Device (Yellow Status)**
   - **Input:**
     - Error Rates: [0, 3, 1]
     - Ages: [2, 5, 7]
   - **Expected Output:** "Yellow"
   - **Result:** Passed
3. **TC-L-03: Three Critical Devices (Red Status)**
   - **Input:**
     - Error Rates: [3, 3, 3]
     - Ages: [10, 11, 12]
   - **Expected Output:** "Red"
   - **Result:** Passed
4. **TC-L-04: Mix of Critical and Non-Critical Devices (Yellow Status)**
   - **Input:**
     - Error Rates: [3, 2, 1]
     - Ages: [10, 7, 5]
   - **Expected Output:** "Yellow"
   - **Result:** Passed
5. **TC-L-05: Edge Case (Transition to Yellow)**
   - **Input:**
     - Error Rates: [2, 3, 2]
     - Ages: [9, 10, 6]
   - **Expected Output:** "Yellow"
   - **Result:** Passed

Mention all obstacles you perhaps faced during the coding session.

1. **Decision Logic Complexity**: Combining device error rates and device ages into a single decision-making logic required careful planning to ensure the station status was classified correctly.
2. **Initial Test Failures**: Some test cases failed during early development, revealing shortcomings in the initial logic and necessitating several refinements.
3. **Maintaining Modularity:** Designing the station status calculation as a modular and reusable function, while still handling multiple conditional checks, required thoughtful structuring to keep the code readable and maintainable.
4. **Balancing Age and Error Thresholds**: Identifying an appropriate balance between the impact of device age and error rate on the overall status involved multiple iterations and adjustments.

5. **Handling Boundary Conditions:** Correctly interpreting edge cases—especially transitions between "Yellow" and "Red" states—required additional debugging and careful validation.

How well did pair programming and test first work for you?

Pair Programming:

1. Pair programming encouraged strong collaboration and idea sharing, allowing logical issues to be detected early and alternative solutions to be discussed.
2. It was particularly useful during debugging and while improving the quality of test cases.
3. Working together improved the clarity and correctness of use case interpretation.
4. Continuous discussion between partners helped cover a wider range of possible scenarios.
5. Boundary conditions were identified more easily, as one developer often clarified uncertainties for the other.
6. Many doubts were resolved through peer feedback, resulting in faster and more accurate handling of edge cases.

Test-First Approach:

1. Continuous testing ensured that newly added conditions behaved as expected, avoiding error accumulation.
2. Defining test cases before coding helped establish clear functional expectations.
3. The Test-First method offered a structured development path aligned with predefined requirements.
4. It ensured that important scenarios were tested early, reducing the chance of failures later in the process.
5. This approach promoted user-oriented thinking, leading to improved overall code quality.

2.2 Unit Test

Systematic Approach for Deriving Test Cases:

The **Boundary Value Analysis (BVA)** and **Equivalence Partitioning (EP)** techniques were applied to derive the unit test cases. These approaches ensure that the station status monitoring unit behaves correctly within valid input ranges, at boundary conditions, and under extreme scenarios. In addition, negative test cases were identified to assess system robustness against invalid or unexpected inputs.

Equivalence partitions were defined based on the number of critical devices (EP-01: none, EP-02: one, EP-03: two, EP-04: three or more) and invalid or empty inputs (EP-05).

High-Level Test Cases:

1. **TC-S-01: All Devices in Non-Critical State (Green Status)**

Input: Error Rates: [1, 1, 1], Ages: [2, 3, 4]
Expected Output: Green
Reason: Verifies correct behavior when no device exceeds critical thresholds (Covers EP-01).

2. **TC-S-02: All Devices in Critical State (Red Status)**
   Input: Error Rates: [3, 3, 3], Ages: [10, 11, 12]
   Expected Output: Red
   Reason: Confirms correct classification when all devices are in a critical condition (Covers EP-04).
3. **TC-S-03: Boundary Case – Transition from Green to Yellow**
   Input: Error Rates: [2, 2, 3], Ages: [8, 8, 8]
   Expected Output: Yellow
   Reason: Tests the boundary where a single critical device triggers a status changes (Covers EP-02).
4. **TC-S-04: Boundary Case – Transition from Yellow to Red**
   Input: Error Rates: [3, 3, 3], Ages: [9, 10, 10]
   Expected Output: Red
   Reason: Verifies escalation when the number of critical devices reaches the threshold (Covers EP-04).
5. **TC-S-05: Maximum Input Scenario**
   Input: [3, 3, 3, 3, 3, 3, 3, 3, 3, 3], Ages: [10, 11, 12, 13, 14, 15, 16, 17, 18, 19]
   Expected Output: Red
   Reason: Ensures Scalability and correct Functionality with the maximum number of devices allowed (Covers EP-04).
6. **TC-S-06: Mixed Ages and Error Rates with Multiple Critical Devices (Red Status)**
   Input: Error Rates: [3, 2, 3], Ages: [5, 10, 12]
   Expected Output: Red
   Reason: Tests correct classification for mixed scenarios using both age and error thresholds (Covers EP-03).
7. **TC-S-07: Minimum Input Scenario**
   Input: Error Rates: [0], Ages: [1]
   Expected Output: Green
   Reason: Verifies correct behavior with the minimum number of devices (Covers EP-01).
8. **TC-S-08: Negative Test Case – Empty Input Arrays**
   Input: Error Rates: [], Ages: []
   Expected Behavior: The system should handle the input gracefully without failure.
   Reason: Evaluates robustness when no devices are provided (Covers EP-05).
9. **TC-S-09: Negative Test Case – Invalid Input Values**
   Input: Error Rates: [-1, 5], Ages: [-2, 200]
   Expected Behavior: Invalid values should not affect correct status classification.
   Reason: Tests robustness against out-of-range input values (Covers EP-05).

Comparison with Test-First Cases:

Overlap: Many test-first cases focus on core scenarios like no critical devices, a single critical device, and multiple critical devices. These overlap with the systematic high-level test cases.

Enhancements: The high-level cases include more diverse edge cases (e.g., old but non- critical devices, extreme edge conditions, and input size limits) that were not covered in the test-first approach.

Systematic Coverage: The high-level test cases ensure wider validation across boundary values and equivalence classes, thereby strengthening overall test completeness and system robustness.

Attachment A

Figure 1: Main.java – Implementation and execution of test cases

The implementation and execution of the test cases are shown in Figure 1

```java
/*
Inputs:
Array of error rates (integer array).
Array of device ages (integer array).
Number of devices (integer).

Output:
Status: Return "Green", "Yellow", or "Red" based on the input values.
*/

public class Main {

    Run | Debug
    public static void main(String[] args) {

        // Test Case 1: No critical devices
        int[] errorRatesCase1 = {0, 0, 0};
        int[] agesCase1 = {2, 5, 7};
        int numDevicesCase1 = 3;

        String expectedStatusCase1 = "Green";
        String actualStatusCase1 =
                StationStatus.calculateStationStatus(errorRatesCase1, agesCase1, numDevicesCase1);

        if (actualStatusCase1.equals(expectedStatusCase1)) {
            System.out.println(x: "Test Case 1 Passed!");
        } else {
            System.out.println(x: "Test Case 1 Failed!");
        }

        // Test Case 2: 1 critical device
        int[] errorRatesCase2 = {0, 3, 1};
        int[] agesCase2 = {2, 5, 7};
        int numDevicesCase2 = 3;

        String expectedStatusCase2 = "Yellow";
        String actualStatusCase2 =
                StationStatus.calculateStationStatus(errorRatesCase2, agesCase2, numDevicesCase2);

        if (actualStatusCase2.equals(expectedStatusCase2)) {
            System.out.println(x: "Test Case 2 Passed!");
        } else {
            System.out.println(x: "Test Case 2 Failed!");
        }
```

```java
        // Test Case 3: 3 critical devices
        int[] errorRatesCase3 = {3, 3, 3};
        int[] agesCase3 = {10, 11, 12};
        int numDevicesCase3 = 3;

        String expectedStatusCase3 = "Red";
        String actualStatusCase3 =
                StationStatus.calculateStationStatus(errorRatesCase3, agesCase3, numDevicesCase3);

        if (actualStatusCase3.equals(expectedStatusCase3)) {
            System.out.println(x: "Test Case 3 Passed!");
        } else {
            System.out.println(x: "Test Case 3 Failed!");
        }

        // Test Case 4: Mix of critical and non-critical devices (Yellow)
        int[] errorRatesCase4 = {3, 2, 1};
        int[] agesCase4 = {10, 7, 5};
        int numDevicesCase4 = 3;

        String expectedStatusCase4 = "Yellow";
        String actualStatusCase4 =
                StationStatus.calculateStationStatus(errorRatesCase4, agesCase4, numDevicesCase4);

        if (actualStatusCase4.equals(expectedStatusCase4)) {
            System.out.println(x: "Test Case 4 Passed!");
        } else {
            System.out.println(x: "Test Case 4 Failed!");
        }

        // Test Case 5: Edge case (transition to Yellow)
        int[] errorRatesCase5 = {2, 3, 2};
        int[] agesCase5 = {9, 10, 6};
        int numDevicesCase5 = 3;

        String expectedStatusCase5 = "Yellow";
        String actualStatusCase5 =
                StationStatus.calculateStationStatus(errorRatesCase5, agesCase5, numDevicesCase5);

        if (actualStatusCase5.equals(expectedStatusCase5)) {
            System.out.println(x: "Test Case 5 Passed!");
        } else {
            System.out.println(x: "Test Case 5 Failed!");
        }
    }
}
```

Figure 2: StationStatus.java – Station status calculation logic

The station status decision logic is implemented as shown in Figure 2.

```java
public class StationStatus {

    public static String calculateStationStatus(int[] errorRates, int[] ages, int numDevices) {

        int criticalCount = 0;

        for (int i = 0; i < numDevices; i++) {
            if (errorRates[i] >= 3 || ages[i] >= 10) {
                criticalCount++;
            }
        }

        if (criticalCount >= 3) {
            return "Red";
        } else if (criticalCount >= 1) {
            return "Yellow";
        } else {
            return "Green";
        }
    }
}
```